

Week 9: Transformers for Computer Vision (ViT and CLIP)

Video Transcripts

Video 9.1: Overview

Transformer models, originally designed for natural language processing have been applied in computer vision in recent years, revolutionising how visual data is processed and analysed. One of the most notable applications is image classifications, where transformers have been adapted to handle image data effectively. Vision Transformer, ViT is a specific adaptation that treats image patches as sequences similar to word tokens in text. By applying self-attention mechanisms, ViT can capture global context and relationships between different parts of an image, leading to impressive performance on image classification benchmarks. Beyond image classification, transformers have been employed in object detection tasks. Traditional object detection methods often rely on region proposals and convolutional neural networks, CNNs, to identify objects within an image. However, transformer based models like detection transformers streamline this process by leveraging self-attention to directly model the relationships between objects and their context within an image. This approach simplifies the detection pipeline and allows for more accurate localisation and classification of objects even in complex things. Transformers are also being explored for generation tasks in computer vision, such as text to image generation, image synthesis and style transfer. Their capacity to model complex data distributions enables the creation of high quality, realistic images and the seamless blending of different styles. As research continues, the application of transformer models in computer vision is likely to expand further, offering new possibilities for innovation in how vision information is processed and utilised.

Video 9.2: Fine-tuning ViTs for Classification - Demo

Because Vision Transformers, ViT, demand expensive data to reach high accuracy levels, generating this data can significantly prolong project timelines. When data availability is limited, leveraging a pre-trained ViT model for fine tuning is often the favourite approach.

These pre-trained models are initially trained on vast data sets of general image content, allowing them to learn robust features. Subsequently, we can fine tune these models using a smaller domain-specific data set. This method enables us to achieve excellent model performance while significantly reducing the computational resources needed.

Let's look at a demo of using a small data set of beans images to fine tuning a pre-trained ViT to build a customised image classifier to differentiate the healthy and unhealthy bean leaves.

First, let's download the data set.

Then we need to process the image data using ViT feature extractor.

When ViT models are trained, specific transformations are applied to the training images.

Therefore, we need to apply the same transformations on our beans images.

To make sure we apply the correct transformations, we use the ViT feature extractor initialised with a configuration that was saved along with the pre-trained model we plan to use.

In our case we'll be using the Google ViT model, so let's load its feature extractor from the Hugging Face Hub.

We write a function to process the data using the loaded feature extractor and we apply this function on all the images in beans data set. Finally, we can train and evaluate our model. Step one we collate all the images into batches. Step two. We need to define an evaluation metric. In this example we use the simplest metric for classification task, accuracy. Step three. We can load the pre-trained vit model from the hub. Step four. Before training our model we need to set up the training configuration by defining training arguments. Now we can pass all the images to trainer and start training or see fine tuning our VIT model. As we can see during the training, we can achieve around 99% training accuracy. How about validation accuracy? We need to pass the validation data to the trainer and evaluate the validation performance. We can achieve 100% validation accuracy. That's great. Lastly, we will apply this trained model on an image from the test data set. This is the image we selected from the test data set. After pass the image to the trained model, we get the prediction probabilities 98.6% for angular leaf spot, 1% for bean rust and 0.4% for healthy. Therefore the final prediction is the leaf in this image is unhealthy with angular leaf spot which is correct. This is a complete process of using a pre-trained VIT model, fine tuning it using our own images and utilise the model to make image classification decisions. Feel free to use your own images to fine tune a VIT model and make it work for you.

Video 9.3: Text-to-Image Generation 1

Besides image classification, transformer models have been extensively utilised in text-to-image applications. In this video and the following three videos, we will explain in detail how the Transformer model is utilised in text-to-image generation tasks. Text-to-image generation focuses on producing state-of-the-art images from a given text prompt. For example, given the input 'A head of broccoli made out of modelling clay, smiling in the sun', we would like the model to be able to output an image that accurately matches the text prompt, as shown in this figure. This is clearly a highly challenging problem. Text understanding and image generation are difficult to solve on their own. Text-to-image generation such as this presents an additional challenge, because the model must also learn how to cross the bridge between the two domains and learn a shared representation that allows it to accurately convert from a block of text to a high-fidelity image without loss of information.

Moreover, in order to be successful, the model must be able to combine concepts and styles that may never have seen before. For example, there are no 'Michelangelo fresco containing people wearing virtual reality headsets', but we would like our model to be able to create such an image if we ask it to. Equally, it will be desirable for the model to accurately infer how objects in the generated image related to each other based on the text prompt. For example, 'A picture of an astronaut riding a donut through space' should look very different from one of 'An astronaut eating a donut in a crowded space.' The model must learn how words are given meaning through context and how to convert explicit textual relationships between entities to images that imply the same meaning. In this and the next few videos, we will use DALL-E 2 from OpenAI as an example to explain how to do text-to-image generation using transformer models. DALL-E 2 is an extremely impressive model that has furthered our understanding of AI's ability to solve these types of multimodal problems.

It not only has ramifications academically, but also forces us to ask big questions relating to the role of AI in creative processes that previously was thought to be unique to humans. Let's start by exploring how DALL-E 2 works. This figure shows an overall architecture of DALL-E 2. There are three distinct parts to consider: the text encoder, the prior, and the decoder. Text is first passed through the text encoder to produce a text embedding vector. This vector is then transformed by the prior to produce an image embedding vector. Finally, this is passed through the decoder, along with the original text, to produce the generated image. We will step through each component to get a complete picture of how DALL-E 2 works in practice.

Video 9.4: Text-to-Image Generation 2

The aim of the text encoder is to convert the text prompt into an embedding vector that represents the conceptual meaning of the text prompt within a latent space. Converting discrete text to a continuous latent space vector is essential for all downstream tasks. And after that, we can continue to manipulate the vector further, depending on our particular goal. In DALL-E 2, the authors do not train the text encoder from scratch, but instead make use of an existing model called Contrastive Language Image Pre-training, CLIP. This is also produced by OpenAI. Therefore, to understand the text encoder, we must first understand how CLIP works. CLIP was unveiled in February 2021 and is described as a neural network that efficiently learns visual concepts from natural language supervision. It uses a technique called contrastive learning to match image with text descriptions. The model is trained on a dataset of 400 million text-image pairs scraped from the Internet. Some example pairs are shown in this figure.

Given an image and a list of possible text descriptions, its task is to find the one that actually matches the image. The key idea behind contrastive learning is simple. We train two neural networks: a text encoder that converts text to a text embedding and an image encoder that converts an image to an image embedding. Then, given a batch of text-image pairs, we compare all text and image embedding combinations using cosine similarity, and train the networks to maximise the score between matching text-image pairs and minimise the score between incorrect text-image pairs. This process is shown in this figure. Both the text encoder and the image encoder are transformers. The image encoder is a Vision Transformer, ViT. The CLIP authors have tested other model architectures, but from this combination, that means transformer text encoder plus vision transformer image encoder, is to produce the best results. What makes CLIP especially interesting is the way it can be used for zero-shot prediction and tasks that it has never been exposed to.

For example, suppose we want to use CLIP to predict the label of a given image in the ImageNet dataset. We can first convert the ImageNet labels into sentences by using a template; for example, a photo of a label as shown in this figure. To predict the label of a given image, we can pass it through the CLIP image encoder and calculate the cosine similarity between the image embedding and all possible text embeddings in order to find the label with the maximum score as shown here. Notice that we do not need to retrain either of the CLIP neural networks for it to be readily applicable to new tasks. It uses language as the common domain through which any set of labels can be expressed. Using this approach, it is possible to show that CLIP performing well across a wide range of image dataset labeling challenges, as shown in this figure. Other models that have been trained on a specific dataset to predict a given set of labels often fail when applied to different dataset with the same labels, because they are highly optimised to the individual dataset on which they were trained.

CLIP is much more robust as it has learned a deep conceptual understanding of full text descriptions and images, rather than just excelling at the narrow task of assigning a single label to a given image in a dataset. But CLIP is simply still a discriminative model or still a classification model. So, how does it help us to build generative models such as DALL-E 2? The answer is that we can take the trained text encoder and use it as one part of a large model such as DALL-E 2. The trained encoder is simply a generalised model for converting text to a text embedding, which should be useful for downstream tasks such as generating images. The text encoder is able to capture a rich conceptual understanding of the text, as it has been trained to be as similar as possible to its matching image embedding counterpart, which is produced only from the paired images. It is, therefore, the first part of the bridge that we need to be able to cross over from the text domain to the image domain.

Video 9.5: Text-to-Image Generation 3

In this video, let's look at the prior part of DALL-E 2. This stage involved converting the text embedding into a CLIP image embedding. The DALL-E 2 authors tried two different methods for training the prior model: an auto-regressive model and the diffusion model. They found that the diffusion approach outperform the auto-regressive model and was more computationally efficient. Let's look at both and see how they differ. An auto-regressive model generates output sequentially by placing an ordering on the output tokens, for example, words, pixels, etc, and conditioning the next token on previous tokens. This idea has been extensively used in recurrent neural networks, transformers and etc. The auto-regressive prior of DALL-E 2 is an encoder-decoder transformer. It is trained to reproduce the CLIP image embedding, given a CLIP text embedding, as shown in this figure. The model is trained on the CLIP text-image pair dataset. You can think of it as a second part of the bridge that we need in order to jump from the text domain to the image domain.

We are converting a vector from the text embedding latent space to the image embedding latent space. The input text embedding is processed by the encoder of the transformer to produce another representation that is fed to the decoder, alongside the current generated output image embedding. The output is generated one element at a time, using teacher forcing to compare the predicted next element to the actual CLIP image embedding. The sequential nature of the generation means that the auto-regressive model is less computational-efficient than the other method, the diffusion prior. Let's look at it now. Diffusion models are fast becoming the go-to choice for generative modelling practitioners alongside transformers. In DALL-E 2, a decoder-only transformer is used as a prior. Train using a diffusion process. The training and generation process is shown in this figure. During training, each CLIP , text and image embedding pairs are first concatenated into a single vector. Then, the image embedding is noised over 1000 time steps until it is indistinguishable from random noise.

The diffusion prior is then trained to predict the denoised image embedding at the previous time step. The prior has access to the text embedding throughout, so it is able to condition its predictions on this information, gradually transforming the random noise into a predicted CLIP image embedding. The loss function is the average mean square error across the noising steps. To generate new image embeddings, we sample a random vector, prepare the relevant text embedding and pass it through the train diffusion prior multiple times. The purpose of the prior is to provide the decoder with a useful representation of the image to be generated, making use of the pre-trained CLIP model. However, it is feasible that this step isn't necessary. Perhaps, we could just pass the text

embedding directly to the decoder instead of the image embedding, or ignore the CLIP embeddings completely and condition only on the text prompt. Would this impact the quality of the generations?

To test this, the DALL-E 2 authors tried three different approaches. First, feed the decoder only with the text prompt and the zero-vector for the image embedding. Second, feed the decoder with the text prompt and the text embedding as if it were an image embedding. Third, feed the decoder with the text prompt and the image embedding. Example results are showing in this figure. We can see that when the decoder is starved of image embedding information, it can only produce a rough approximation of the text prompt, missing key information such as the calculator. Using the text embedding as if it were an image embedding performs slightly better, though it is not able to capture the relationship between the hedgehog and the calculator. Only the full model with the prior produces an image that accurately reflects all of the information contained within the prompt.

Video 9.6: Module Summary

The vision transformer, ViT, has emerged as a strong contender to convolutional neural networks, CNNs. CNNs have long been the standard in computer vision and are extensively used for image recognition tasks. ViT models offer significant advantages over traditional CNNs, achieving greater computational efficiency and accuracy. While CNNs have dominated computer vision for years, vision transformers have demonstrated exceptional capabilities on various tasks such as image classification, text to image generation, and many other vision tasks we don't have time to cover in this module. For example, image segmentation, anomaly detection, action recognition, and autonomous driving. However, transformers generally require large data sets to achieve high accuracy, which can lengthen the data collection phase of a project. In scenarios with limited data, CNNs typically performed more effectively than transformers. Despite this, transformers often have shorter training times compared to CNNs. Therefore, when model training time is constrained, transformers might be the preferred choice due to their computational efficiency and accuracy. Furthermore, the self-attention mechanism inherent in transformers enhances model interpretability. It allows for the visualisation of attention maps which can provide insights into the model's focus areas and guide developers in identifying and addressing model weakness. This is something that is often challenging with CNNs. All in all, this is a thrilling period for advancements in natural language processing and computer vision, transformer models are proving to be highly effective in both domains and are facilitating their integration, leading to numerous innovative and creative AI applications.